# MediCare RAG Assistant

## Technical Documentation

*Engineering Reference for the RAG-Based Healthcare Customer Support System*

**Stack: FastAPI · React · LangGraph · ChromaDB · GPT-4o**

Domain: Healthcare & Insurance Customer Support

*Version 1.0 | Innomatics Research Labs Internship*

# 1. Introduction

## 1.1 What is RAG?

Retrieval-Augmented Generation (RAG) is an AI architecture pattern that combines two fundamentally different capabilities: (1) the factual grounding of information retrieval systems and (2) the language fluency of large language models (LLMs). Rather than relying solely on the LLM's parametric memory — which can be outdated, hallucinated, or too generic — RAG dynamically fetches relevant documents from an external knowledge source at inference time and injects them into the LLM's context window.

The core insight of RAG is that an LLM's role should be a reasoning engine, not a knowledge store. The knowledge should live in a versioned, queryable vector database that can be updated without retraining the model.

**RAG vs. Fine-Tuning**

Fine-tuning bakes knowledge into model weights — expensive, hard to update, risk of catastrophic forgetting. RAG keeps knowledge external and updatable — add a new policy PDF and the system immediately knows it. For enterprise knowledge bases that change frequently (insurance policies, drug formularies), RAG is clearly superior.

## 1.2 Why RAG for Healthcare Support?

- Policy documents change quarterly — RAG allows zero-downtime knowledge updates
- LLMs hallucinate medical/financial details — RAG grounds answers in authoritative sources
- Regulatory compliance requires traceable, citable answers — RAG provides source documents
- Sensitive domains require careful escalation — RAG confidence scores enable HITL routing

## 1.3 Use Case Overview

A healthcare insurance company deploys this system as the first line of customer support. Users upload their policy PDFs, plan documents, or FAQ guides into the knowledge base. Customers then interact via the chat interface to get instant, accurate answers about coverage, claims, billing, and plan comparisons. When the system is uncertain or detects a sensitive query, it creates a ticket for human agent review.

# 2. System Architecture Explanation

## 2.1 Component Interaction Map

The system has three major interaction layers: client, server, and AI+data. Here is how they interconnect:

```
CLIENT LAYER
  React SPA (Vite, Tailwind, Framer Motion)
       │ Axios HTTP calls to /api/v1/*
       ▼
SERVER LAYER
  FastAPI (Uvicorn ASGI)
  ├── Pydantic request/response validation
  ├── CORS middleware for local development
  └── Routes: /chat, /ingest, /hitl/*, /status
       │
       ▼
AI PIPELINE LAYER (LangGraph)
  input_node ──▶ retrieve_node ──▶ [Router]
                                     ├── generate_node  ──▶ END
                                     └── escalate_node  ──▶ END
       │                  │
       ▼                  ▼
  OpenAI API          ChromaDB
  (GPT-4o + Embeddings)  (Local Vector Store)
```

## 2.2 Detailed Component Interactions

| Interaction | Detail |
|---|---|
| React → FastAPI | Axios POST with JSON body. Session ID maintained in sessionStorage. Conversation history sent with each request. |
| FastAPI → LangGraph | Async function call to run_rag_pipeline(). Graph runs synchronously inside async wrapper. |
| LangGraph → OpenAI | Two calls per turn: embed_query() via LangChain, then chat completion via ChatOpenAI. Both use the same API key. |
| LangGraph → ChromaDB | collection.query() with pre-computed query embedding. Returns documents + distances. |
| FastAPI → HITL | Direct Python function call to hitl_service.create_ticket(). No async needed (in-memory). |

# 3. Design Decisions

## 3.1 Chunk Size Choice: 800 Characters

The 800-character chunk size was determined by analyzing typical insurance policy clause lengths. A single coverage clause or exclusion paragraph in an insurance policy averages 600–900 characters. Smaller chunks (200–400 chars) lose the full context of a clause. Larger chunks (1500+ chars) reduce retrieval precision because a chunk may cover multiple unrelated topics.

The 150-character overlap (19% of chunk size) ensures that sentences split across chunk boundaries are still retrievable. This is critical for medical terms that appear at the start or end of a paragraph.

## 3.2 Embedding Strategy

text-embedding-3-small was chosen over text-embedding-ada-002 for the following reasons:

| Model | Key Facts |
|---|---|
| text-embedding-ada-002 | 1536 dims, $0.10/1M tokens, older architecture |
| text-embedding-3-small | 1536 dims, $0.02/1M tokens (5x cheaper), higher MTEB scores |
| text-embedding-3-large | 3072 dims, $0.13/1M tokens, marginal gain for this use case |

For a healthcare support bot, 3-small provides sufficient semantic understanding of medical/insurance terminology at optimal cost. The same model is used for both indexing and querying, ensuring embedding space consistency.

## 3.3 Retrieval Approach

Cosine similarity was chosen over L2 (Euclidean) distance because it measures angular similarity — direction matters more than magnitude for semantic embeddings. The confidence score (1 − cosine_distance) provides an intuitive 0-to-1 range that directly maps to the HITL escalation threshold.

Top-K = 5 retrieves enough context to cover multi-part queries (e.g., 'what is my deductible and does it apply to prescriptions?') without overloading the GPT-4o context window.

## 3.4 Prompt Design Logic

The RAG system prompt was designed with four constraints in mind:

1. Answer from context only — prevents hallucination of policy details
2. Empathetic, professional tone — appropriate for healthcare domain
3. No medical diagnoses or legal advice — liability protection
4. Direct users to official channels for sensitive topics — regulatory compliance

The intent classification prompt uses zero-temperature (deterministic) generation with a closed-vocabulary output (model returns only the label). This eliminates parsing errors and ensures consistent routing.

# 4. Workflow Explanation

## 4.1 LangGraph Usage

LangGraph was selected over a simple sequential pipeline for three key reasons:

5. Explicit control flow — the graph topology is inspectable and testable as a directed graph
6. Conditional routing — branching to escalation is a first-class feature, not a monkeypatched if-statement
7. Extensibility — adding a re-ranker node, a query rewriting node, or a multi-document router requires only adding nodes/edges to the existing graph

## 4.2 Node Responsibilities

**input_node**

Responsibility: Understand what the user is asking before retrieving anything.

Why first? Retrieval can be tailored per intent (e.g., billing queries may need different top-K).

Failure mode: LLM timeout → defaults to general_info, pipeline continues gracefully.

**retrieve_node**

Responsibility: Find the most relevant policy document chunks.

Why second? Intent is not used in retrieval currently (pure semantic search), but the node placement allows future intent-conditional retrieval.

Output: confidence score determines whether generation or escalation is appropriate.

**generate_node**

Responsibility: Synthesize a grounded, conversational answer from retrieved context.

Conversation history injection: last 6 messages (3 turns) provide follow-up context.

Temperature 0.3: slight creativity allows natural language variation without hallucination risk.

**escalate_node**

Responsibility: Protect users from unreliable answers by routing to human agents.

No LLM call: escalation messages are template-based to ensure consistency and speed.

Creates a structured ticket with full context for the human agent.

# 5. Conditional Logic

## 5.1 Intent Detection

The seven intent categories cover the full scope of healthcare support interactions:

| Intent | Typical Queries |
|---|---|
| general_info | What plans do you offer? How does deductible work? |
| claim_status | What is the status of my claim? Why was my claim denied? |
| coverage_question | Is my prescription covered? Does my plan cover dental? |
| billing | Why is my premium higher? I received an unexpected bill. |
| emergency | I need emergency medical help. This is urgent. |
| complex | Multi-part queries, policy comparisons, appeals processes. |
| out_of_scope | Weather questions, unrelated topics. |

## 5.2 Routing Decisions

The routing function implements a priority-ordered decision chain. Emergency and sensitive-keyword detection takes absolute priority because no amount of confidence should override safety. Low confidence is checked second because a high-confidence answer on an out-of-scope question is still wrong. Missing context is the last check because partial context may still produce a useful answer.

**Design Principle: Fail Safe**

When in doubt, escalate to a human. A false positive escalation (unnecessary ticket) is far less harmful than a false negative (wrong medical/financial advice given confidently).
The confidence threshold (0.65) is deliberately conservative for the healthcare domain.

# 6. HITL Implementation

## 6.1 Role of Human Intervention

HITL serves as the safety net for the AI system. No matter how capable the RAG pipeline is, there will always be queries that require human judgment: legal disputes, complex appeals, emotionally distressed users, or situations where even a slightly wrong answer has serious consequences.

The HITL module deliberately has a low threshold for escalation in v1.0. As the system accumulates data on which query types reliably produce good answers, the threshold can be raised and the escalation list can be narrowed.

## 6.2 Benefits of the Current HITL Design

- Zero latency for the user when escalated — template response is immediate
- Human agents have full context (original query + top-3 relevant chunks) to craft a response
- Ticket IDs allow users to follow up on their escalated queries
- Ticket data is a valuable corpus for identifying coverage gaps in the knowledge base

## 6.3 Limitations

- In-memory ticket store does not survive application restarts (production needs PostgreSQL)
- No notification system — agents must poll the dashboard (production needs webhooks/email)
- No SLA tracking — ticket age is displayed but no automatic escalation after N hours
- Human response is not fed back into the vector store (future: retrieval-augmented HITL)

# 7. Challenges & Trade-offs

| Challenge / Trade-off | Resolution / Decision |
|---|---|
| Retrieval accuracy vs speed | HNSW approximate search is fast (~5ms) with minimal accuracy loss vs exact search for this corpus size |
| Chunk size vs context quality | 800 chars balances clause completeness vs retrieval precision; configurable for experimentation |
| Cost vs performance | text-embedding-3-small (5x cheaper than ada-002) with no measurable quality drop for insurance text |
| Latency vs conversation depth | Only last 3 turns of history injected; balances context with GPT-4o latency |
| Escalation sensitivity vs user experience | Conservative threshold (0.65) prioritizes safety; tunable once sufficient feedback data exists |
| LLM temperature: creativity vs accuracy | Temperature 0.3 for generation; 0.0 for classification to balance fluency and determinism |
| Single vs multi-document retrieval | v1.0 uses a single merged collection; v1.1 will add per-document namespacing |

# 8. Testing Strategy

## 8.1 Testing Approach

Testing covers four layers: unit tests for individual services, integration tests for the API, end-to-end tests for the full LangGraph pipeline, and manual testing with domain-specific queries.

## 8.2 Sample Test Queries by Category

| Category | Sample Queries |
|---|---|
| Coverage (should answer) | Is mental health therapy covered? What is my annual deductible? |
| Claims (should answer) | How do I submit a claim? What documents do I need for reimbursement? |
| Billing (should answer) | Why did my premium increase? Can I get a payment plan? |
| Low confidence (should escalate) | What are the hidden charges in my plan? [vague query] |
| Sensitive keyword (should escalate) | I want to sue the hospital. This is a legal matter. |
| Emergency (should escalate) | I'm having a heart attack, what do I do? |
| Out of scope (intent detected) | What is the weather in Mumbai tomorrow? |

## 8.3 Retrieval Quality Metrics

- Precision@K: Fraction of retrieved chunks that are relevant to the query
- Confidence score distribution: Should cluster above 0.7 for in-scope queries
- Escalation rate: Track what % of queries are escalated over time

# 9. Future Enhancements

## 9.1 Multi-Document Support

Add a document registry with namespaced ChromaDB collections per document. Allow users to query across multiple PDFs or restrict retrieval to a specific plan document. This enables side-by-side plan comparisons.

## 9.2 Feedback Loop

Implement a thumbs up/down UI on each response. Log (query, retrieved_chunks, answer, rating) to a PostgreSQL feedback table. Use low-rated responses to identify retrieval failures and refine the chunking strategy or prompts.

## 9.3 Memory Integration

Replace the simple conversation history list with Redis-backed persistent memory using LangChain's RedisChatMessageHistory. This allows users to resume sessions across page refreshes and enables longitudinal support conversations.

## 9.4 Query Rewriting Node

Add a pre-retrieval node that rewrites ambiguous queries into more search-friendly forms (e.g., 'does my plan cover this' → 'prescription drug coverage policy'). This significantly improves retrieval precision for conversational queries.

## 9.5 Deployment

- Containerize with Docker Compose (FastAPI + ChromaDB volumes + React Nginx)
- Deploy FastAPI on AWS ECS or Google Cloud Run (serverless)
- Swap ChromaDB for Pinecone (managed, globally distributed)
- Add LangSmith tracing for production observability and debugging
- Implement RBAC so agents can only see tickets from their assigned queue